

Unit - 9 : Class in Python

1. Introduction

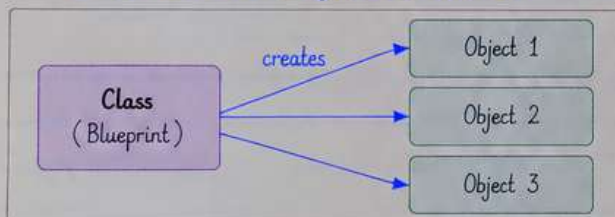
- Class in Python एक blueprint (खाका) होता है, जिसके द्वारा हम objects (वस्तु) create करते हैं।
- यह real world entities को model करने के लिए use होता है।
- Class में data (variables) और functions (methods) दोनों होते हैं, जो मिलकर object का behavior define करते हैं।
- Python एक object-oriented programming language है, इसलिए almost सभी चीजें class और object के रूप में represent होती हैं।

2. Why Use Class ?

- Code को organize करने के लिए।
- Real world problems को model करने के लिए।
- Code reusability बढ़ाने के लिए।
- Data को सुरक्षित रखने (data hiding) के लिए।
- Large programs को manage करने के लिए।

3. Class and Object

- **Class** : यह एक template होता है जिसमें attributes और methods define होते हैं।
- **Object** : Class का instance (वास्तविक रूप) होता है।
- हम एक class से कई objects बना सकते हैं।



4. Syntax of Class

```
class ClassName:           ← class definition
    def __init__(self, parameters): ← constructor (special method)
        # initialize attributes
        self.attribute = value      ← instance variable

    def method_name(self): ← method
        # method definition
        pass
```

5. Example : Creating a Class and Object

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(f"Name : {self.name}")
        print(f"Age : {self.age}")

# creating object
s1 = Student("Rahul", 20)
s1.display()
```

6. Output

Output :

Name : Rahul
Age : 20

7. __init__ () Method (Constructor)

- `__init__()` एक special method (constructor) होता है जो object create होते समय automatically call होता है।
- इसका use object के initial values set करने के लिए किया जाता है।
- यह method का नाम हमेशा `__init__` होता है।

8. self Keyword

- `self` एक reference variable है जो current object को represent करता है।
- इसका use class के attributes और methods को access करने के लिए होता है।
- हर method में `self` को first parameter के रूप में लिखना होता है।

9. Types of Variables in Class

Type	Description	Example
Instance Variable	यह हर object के लिए अलग होता है।	<code>self.name</code>
Class Variable	यह सभी objects के लिए common होता है।	<code>school = "ABC"</code>
Local Variable	यह method के अंदर define होता है और केवल उसी method में use होता है।	<code>x = 10</code>

10. Example : Class and Instance Variable

```
class Student:
    school = "ABC College" # class variable
    def __init__(self, name):
        self.name = name # instance variable
    def show(self):
        print(f"Name : {self.name}")
        print(f"School : {Student.school}")

s1 = Student("Vishal")
s2 = Student("Aman")
s1.show()
s2.show()
```

Output :

Name : Vishal
School : ABC College

Name : Aman
School : ABC College

11. Key Points

- Class एक blueprint है और object उसका instance होता है।
- `__init__()` constructor object के initial values set करता है।
- `self` current object को refer करता है।
- Class variable सभी objects के लिए common होता है।
- Instance variable हर object के लिए अलग होता है।
- Class का use करने से code organized, reusable और maintainable बनता है।

Conclusion : Class in Python हमें real world problems को model करने का एक powerful और flexible तरीका प्रदान करता है।

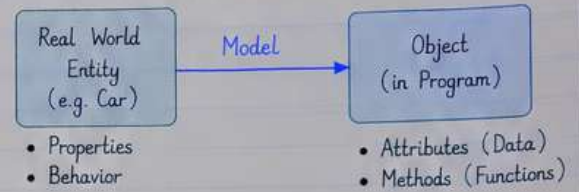
Topic : Principles of Object Orientation

1. Introduction

- Object Oriented Programming (OOP) एक programming paradigm है जो real world की problems को solve करने के लिए use होता है।
- OOP में program को objects के रूप में organize किया जाता है, जो data (attributes) और behavior (methods) को combine करते हैं।
- इसके main principles कुछ basic concepts हैं जिनके द्वारा OOP काम करता है।

"Object Oriented Programming makes code more modular, reusable and easy to maintain."

Real World to OOP



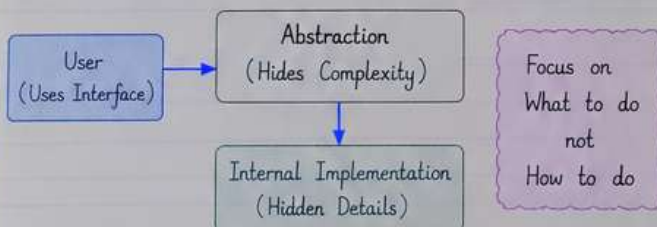
2. Main Principles of OOP

- OOP के चार main principles होते हैं :

1. Encapsulation
 2. Abstraction
 3. Inheritance
 4. Polymorphism
- ये सभी principles मिलकर code को modular, secure, flexible और reusable बनाते हैं।

4. Abstraction

- Abstraction का मतलब है केवल जरूरी (important) details को दिखाना और unnecessary details को hide करना।
- यह user को simple interface देता है।
- Abstraction के लिए हम abstract class और interface का use करते हैं (Python में abstract class के लिए abc module होता है)।



3. Encapsulation

- Encapsulation का मतलब है data (variables) और methods को एक single unit (class) में bind करना।
- यह data को protect करता है और direct access को restrict करता है।
- इसके लिए हम access modifiers (public, private, protected) का use करते हैं।

```
class Student:
    def __init__(self, name, age):
        self.__name = name # private variable
        self.__age = age # private variable
    def get_name(self): # getter method
        return self.__name
```

Data Hiding
Security
Better Control

5. Inheritance

- Inheritance का मतलब है एक class (child class) का दूसरी class (parent class) के properties और methods को प्राप्त करना।
- यह code reusability को बढ़ाता है।
- Child class अपने parent class के behavior को extend भी कर सकती है (नई functionality add करके)।

```
class Animal:
    def sound(self):
        print("Animal makes a sound")
class Dog(Animal): # Inheritance
    def sound(self): # Method Overriding
        print("Dog barks")
```

Code Reuse
Extend
Modify

6. Polymorphism

- Polymorphism का मतलब है "one interface, many forms"।
- एक ही method अलग-अलग objects के लिए अलग behavior कर सकता है।
- यह method overriding और method overloading (कुछ languages में) के द्वारा achieve होता है।
- Python में method overloading directly नहीं होता, but हम default arguments का use करके similar behavior achieve कर सकते हैं।

```
class Cat:
    def sound(self):
        print("Cat meows")
class Cow:
    def sound(self):
        print("Cow moos")
# Same method name, different behavior
def make_sound(animals):
    animals.sound()
make_sound(Cat())
make_sound(Cow())
```

Same Interface
Different Behavior
Flexibility

7. Conclusion

Principles of Object Orientation (Encapsulation, Abstraction, Inheritance, Polymorphism) मिलकर OOP को एक powerful programming approach बनाते हैं, जो real world applications को develop करने के लिए बहुत useful होता है।

Topic : Creating Classes, Instance Methods

1. Creating Classes

- Class एक blueprint (स्काफ) होता है जिसके द्वारा हम objects create करते हैं।
- Class में हम attributes (data) और methods (functions) define करते हैं जो उसके objects के behavior को तय करते हैं।
- class का use real world entities को model करने के लिए होता है।
- Python में class define करने के लिए class keyword का use किया जाता है।

2. Syntax of Creating a Class

```
class ClassName:  
    def __init__(self, parameters):  
        # initialize attributes  
        self.attribute = value
```

3. Example : Creating a Class

```
class Student:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age  
  
    def display(self):  
        print(f"Name : {self.name}")  
        print(f"Age : {self.age}")
```

4. Creating Objects (Instances)

- Class से object (instance) create करने के लिए class name को call किया जाता है।
- Object बनाने के बाद हम उसके methods को access कर सकते हैं।

```
# creating objects  
s1 = Student("Rahul", 20)  
s2 = Student("Priya", 19)  
  
# calling method  
s1.display()  
s2.display()
```

Output :

```
Name : Rahul  
Age : 20  
  
Name : Priya  
Age : 19
```

Note :

- हर object के अपने अलग attributes होते हैं।
- एक ही class से कई objects बना सकते हैं।

5. Instance Methods

- Instance method एक ऐसा method होता है जो object के साथ काम करता है और उसके data (attributes) को access करता है।
- Instance method का पहला parameter हमेशा self होता है, जो current object को refer करता है।
- इस method का use object के behavior को define करने के लिए होता है।

6. Syntax of Instance Method

```
class ClassName:  
    def method_name(self, parameters):  
        # method body  
        # use self to access instance variables  
        pass
```

7. Example : Instance Methods

```
class Rectangle:  
    def __init__(self, length, width):  
        self.length = length  
        self.width = width  
  
    def area(self): # instance method  
        return self.length * self.width  
  
    def display(self):  
        print(f"Length : {self.length}")  
        print(f"Width : {self.width}")  
        print(f"Area : {self.area()}")
```

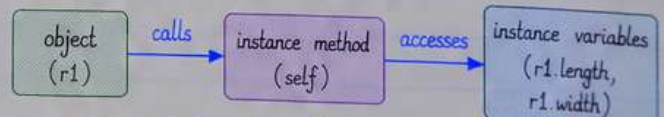
8. Creating Object and Calling Methods

```
r1 = Rectangle(5, 3)  
r1.display()  
  
r2 = Rectangle(10, 4)  
r2.display()
```

Output :

Length : 5	Length : 10
Width : 3	Width : 4
Area : 15	Area : 40

9. How Instance Method Works ?



- Object method को call करता है।
- self उस object को refer करता है।
- Method उस object के data को access करके operation perform करता है।

10. Key Points

- Class एक template है, objects उसके instances होते हैं।
- Instance method हमेशा self को पहला parameter लेता है।
- Instance method object के data को access और manipulate करता है।
- Methods के द्वारा हम object के behavior को define करते हैं।

Topic : File Organization

1. Introduction

- File Organization का मतलब है data को files में किस तरह से store, arrange और manage किया जाता है।
- यह एक method है जिसके द्वारा data को storage device (जैसे Hard Disk, SSD आदि) में logically organize किया जाता है।
- इसका main purpose data को efficiently store करना और later use के लिए easily access करना है।
- Proper file organization से storage space का better use होता है और data को search, read, update और maintain करना आसान हो जाता है।

2. File, Record and Field

- File :** Related data का collection जो एक नाम के साथ store होता है।
- Record :** File में store एक single data unit (जैसे एक student का पूरा data) होता है।
- Field :** Record का एक part होता है जो single information (जैसे name, age) को represent करता है।
- उदाहरण :

File : Student.txt

Roll No	Name	Age	Branch
101	Rahul	20	CSE
102	Priya	19	IT
103	Aman	21	ECE

← Field

← Record

← File (Collection of Records)

3. Need for File Organization

- Data को systematic तरीके से store करने के लिए।
- Data को जल्दी access करने के लिए।
- Storage space का efficient use करने के लिए।
- Data के update, insert और delete को आसान बनाने के लिए।
- Large amount of data को manage करने के लिए।
- Data redundancy और inconsistency को कम करने के लिए।

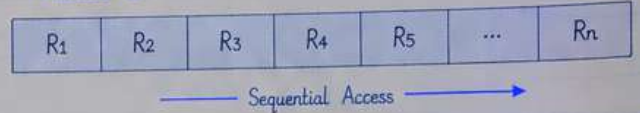
4. Types of File Organization

- Data को store करने के लिए अलग-अलग file organization methods use किए जाते हैं। मुख्य प्रकार निम्नलिखित हैं :

1. Sequential File Organization
2. Direct (Random) File Organization
3. Indexed File Organization
4. Hashed File Organization

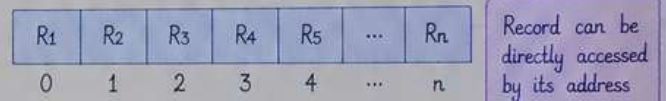
5. Sequential File Organization

- इस method में records को एक fixed order में store किया जाता है (जैसे roll number के increasing order में)।
- Data को read करने के लिए file को beginning से sequentially access करना पड़ता है।
- यह method simple होता है और small data के लिए suitable है।



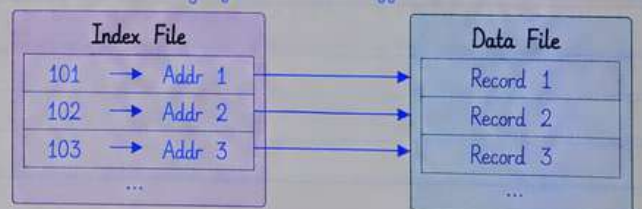
6. Direct (Random) File Organization

- इस method में हर record का एक unique address होता है।
- किसी भी record को directly access किया जा सकता है, बीच के records को read करने की जरूरत नहीं होती।
- यह method large data के लिए fast access प्रदान करता है।



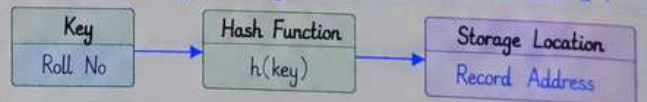
7. Indexed File Organization

- इस method में एक index file maintain की जाती है जो हर record की location (address) को store करती है।
- Index की मदद से required record को जल्दी find किया जा सकता है।
- यह method large files के लिए efficient होता है।



8. Hashed File Organization

- इस method में एक hash function का use करके record की storage location determine की जाती है।
- यह method बहुत fast access प्रदान करता है।
- यह mainly large and dynamic data के लिए use होता है।



9. Advantages of File Organization

- Data को efficiently store और manage करना।
- Fast data access और retrieval।
- Storage space का better utilization।
- Data consistency और integrity maintain करना।
- Large data को handle करने में मदद मिलना।

Conclusion :

File organization एक महत्वपूर्ण concept है जो data को systematic, efficient और easily accessible बनाने में मदद करता है।

Topic : Special Methods

1. Introduction

- Special Methods (या Dunder Methods) Python में ऐसे predefined methods होते हैं जो double underscore (__) से start और end होते हैं।
- इनका use, objects के behavior को define करने के लिए किया जाता है।
- ये methods Python द्वारा automatically call होते हैं जब हम built-in functions या operators का use करते हैं।
- Special methods की मदद से हम अपने class के objects को Python की built-in functionalities के साथ work करने के लिए customize कर सकते हैं।

Example :

```
class Book:
    def __init__(self, title, price):
        self.title = title
        self.price = price

    def __str__(self):
        return f"{self.title} (₹{self.price})"

    def __repr__(self):
        return f"Book('{self.title}', {self.price})"

b = Book("Python", 450)
print(b)
print(repr(b))
```

2. Syntax

```
class ClassName:
    def __method__(self, parameters):
        # method body
        pass
```

Output :

```
Python (₹450)
Book('Python', 450)
```

3. Common Special Methods

Method	Purpose / Description
<code>__init__()</code>	Constructor, object initialize करने के लिए
<code>__str__()</code>	Object की string representation (print)
<code>__repr__()</code>	Official string representation (debugging)
<code>__len__()</code>	<code>len()</code> function के लिए
<code>__add__()</code>	<code>+</code> operator के लिए
<code>__eq__()</code>	<code>==</code> operator के लिए
<code>__lt__()</code>	<code><</code> operator के लिए
<code>__call__()</code>	Object को function की तरह call करने के लिए
<code>__del__()</code>	Object delete होने पर cleanup के लिए

6. __len__() Method

- यह method `len()` function के साथ काम करता है।
- इसका use object की length या size बताने के लिए किया जाता है।

Example :

```
class MyList:
    def __init__(self, items):
        self.items = items

    def __len__(self):
        return len(self.items)

m = MyList([1, 2, 3, 4, 5])
print(len(m))
```

Output :

```
5
```

4. __init__() Method (Constructor)

- यह special method object create होते समय automatically call होता है।
- Iska use object ke initial values set करने के लिए किया जाता है।

Example :

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

s1 = Student("Rahul", 20)
print(s1.name, s1.age)
```

Output :

```
Rahul 20
```

Example :

```
class Number:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return Number(self.value + other.value)

    def __str__(self):
        return str(self.value)

n1 = Number(10)
n2 = Number(5)
n3 = n1 + n2
print(n3)
```

Output :

```
15
```

5. __str__() and __repr__()

- `__str__()`: user के लिए readable string return करता है (print() में use होता है)।
- `__repr__()`: object का official representation return करता है (mainly debugging के लिए)।

7. Operator Overloading (e.g. __add__())

- Special methods की मदद से हम Python के operators (जैसे +, -, ==) के behavior को customize कर सकते हैं।

8. __del__() Method (Destructor)

- यह method object delete होने से पहले automatically call होता है।
- Iska use cleanup activities (जैसे file close करना आदि) के लिए किया जा सकता है।

Key Point :

- Special methods हमें अपने objects को Python की built-in functionalities के साथ integrate करने में मदद करते हैं।
- इनका सही use code को अधिक readable, flexible और powerful करता है।

Topic : Class Variables

1. Introduction

- Class Variable एक ऐसा variable होता है जो class के level पर define किया जाता है, न कि किसी specific object (instance) के लिए।
- यह सभी objects के लिए common होता है।
- Class variable को class name के द्वारा access किया जाता है, लेकिन इसे object के द्वारा भी access किया जा सकता है।
- इसका उपयोग उन data के लिए किया जाता है जो सभी instances के लिए same (common) होने चाहिए।
- यह memory की बचत करता है क्योंकि इसकी केवल एक copy class में बनती है, जो सभी objects के बीच share होती है।

2. Syntax

```
class ClassName:
    class_variable = value # class variable
    def __init__(self, ...):
        self.instance_variable = value # instance variable

# Access
print(ClassName.class_variable)
print(obj.class_variable)
```

3. Example : Class Variable

```
class Student:
    school = "ABC College" # class variable
    def __init__(self, name, age):
        self.name = name # instance variable
        self.age = age # instance variable

    def display(self):
        print(f"Name : {self.name}")
        print(f"Age : {self.age}")
        print(f"School : {Student.school}")
```

4. Output

```
s1 = Student("Rahul", 20)
s2 = Student("Priya", 19)

s1.display()
s2.display()
print(Student.school)
print(s1.school)
print(s2.school)
```

Output :

Name : Rahul
Age : 20
School : ABC College

Name : Priya
Age : 19
School : ABC College

ABC College
ABC College
ABC College

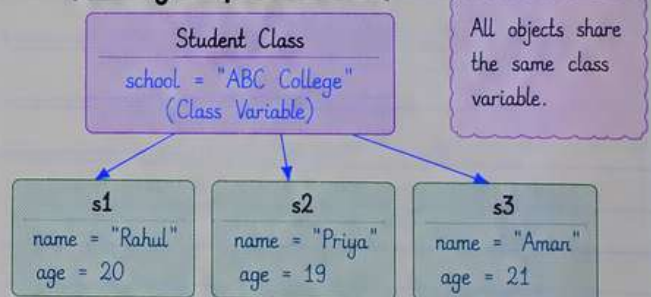
Class Variable vs Instance Variable

Feature	Class Variable	Instance Variable
Defined in	Inside class (outside methods)	Inside method (usually <code>__init__</code>)
Common to	All objects	Single object
Declaration	Directly in class	Using <code>self</code> in method
Memory	Single copy	Separate copy for each object
Access	<code>ClassName.var</code> or <code>obj.var</code>	<code>obj.var</code> (using <code>self</code>)
Use case	Common data (e.g. count)	Individual data (e.g. name, age)

5. How it Works ?

- जब class create होती है, तब class variable memory में एक बार बनता है।
- सभी objects उसी class variable को share करते हैं।
- अगर किसी object के द्वारा class variable को modify किया जाता है, तो वह change सभी objects पर effect करेगा (जब तक कि उसी नाम का instance variable न बन जाए)।
- यदि किसी instance में उसी नाम का instance variable बना दिया जाता है, तो वह class variable को उस object के लिए override कर देता है।

6. Memory Representation



7. Modifying Class Variable

```
Student.school = "XYZ University" # change using class name
print(s1.school) # XYZ University
print(s2.school) # XYZ University
```

8. Key Points

- Class variable सभी objects के लिए common होता है।
- इसे class के अंदर define किया जाता है (methods के बाहर)।
- इसे `ClassName` या object दोनों के द्वारा access किया जा सकता है।
- यह तब उपयोगी होता है जब हमें सभी instances के लिए same data maintain करना हो।
- अगर instance में उसी नाम का variable बना दिया जाता है, तो वह class variable को override कर देता है (सिर्फ उसी instance के लिए)।

Conclusion :

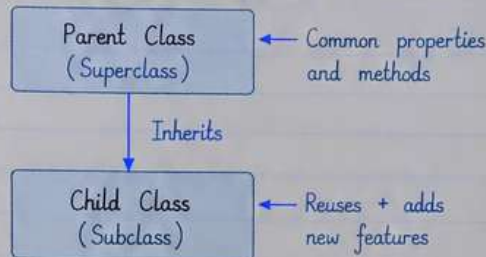
Class variable class के सभी objects के लिए common data provide करता है और memory की efficient use में मदद करता है।

Topic : Inheritance, Polymorphism

1. Inheritance (उत्तराधिकार)

- Inheritance OOP का एक important concept है जिसमें एक class (child class / subclass) दूसरी class (parent class / superclass) की properties और methods को inherit (प्राप्त) करती है।
- इसका use code reusability के लिए होता है।
- Child class, parent class के existing members का use कर सकती है और अपने नए members भी add कर सकती है।
- यह "is-a" relationship को represent करता है।

Inheritance Hierarchy



2. Syntax (Single Inheritance)

```
class Parent:
    def method1(self):
        pass

class Child(Parent): # Inherits from Parent
    def method2(self):
        pass
```

3. Example : Inheritance

```
class Animal:
    def sound(self):
        print("Animal makes a sound")

class Dog(Animal): # Dog inherits from Animal
    def bark(self):
        print("Dog barks")

d = Dog()
d.sound() # inherited method
d.bark() # child class method
```

Output :

Animal makes a sound
Dog barks

4. Types of Inheritance

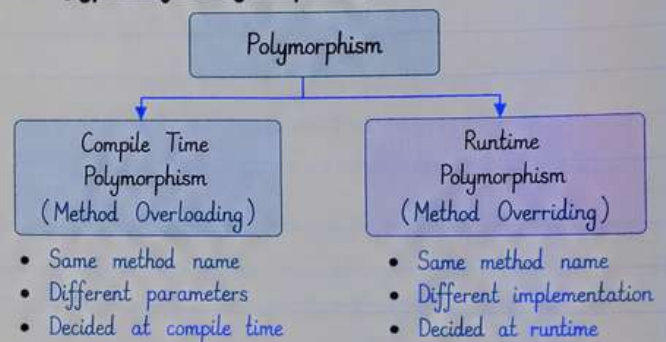
1. Single Inheritance - एक parent, एक child
2. Multiple Inheritance - एक से अधिक parent
3. Multilevel Inheritance - chain की तरह (A → B → C)
4. Hierarchical Inheritance - एक parent, कई child
5. Hybrid Inheritance - combination of multiple types

5. Polymorphism (बहुरूपता)

- Polymorphism का अर्थ है "one interface, many forms" यानी एक ही method अलग-अलग objects के लिए अलग behavior दिखा सकता है।
- यह same method name के साथ different behavior provide करता है।
- Polymorphism code को flexible और extensible बनाता है।
- यह runtime (method overriding) और compile time (method overloading) दोनों में हो सकता है।
- यह "same interface, different implementation" के concept पर काम करता है।

"Polymorphism allows the same function to behave differently in different situations."

6. Types of Polymorphism



7. Example : Method Overloading (Compile Time)

```
class Calculator:
    def add(self, a, b):
        return a + b

    def add(self, a, b, c):
        return a + b + c
```

Same method name "add()" but different number of parameters. (Python does not support true overloading, this is conceptual example)

8. Example : Method Overriding (Runtime)

```
class Animal:
    def sound(self):
        print("Animal makes a sound")

class Cat(Animal):
    def sound(self): # overriding
        print("Cat meows")

a = Animal()
c = Cat()
a.sound()
c.sound()
```

Same method name "sound()" but different implementation in child class.

Output :

Animal makes a sound
Cat meows

9. Key Points

- Inheritance code reusability के लिए होता है।
- Polymorphism same interface के साथ different behavior provide करता है।
- Inheritance "is-a" relationship है, Polymorphism "many forms" देता है।
- Polymorphism का use flexibility और scalability के लिए किया जाता है।
- Dono OOP के important concepts हैं और real world applications में बहुत useful हैं।

Topic : Example 1 of Class in Python

1. Objective

- Class in Python का basic example समझना ।
- Object बनाना और उसके attributes को access करना ।
- Instance method को define और call करना ।
- यह देखना कि class हमें real world entities को model करने में कैसे मदद करती हैं ।

2. Program : (Example 1)

```
class Student:                                # class definition
    def __init__(self, name, age, course):    # constructor
        self.name = name                    # instance variable
        self.age = age
        self.course = course

    def display(self):                        # instance method
        print(f"Name : {self.name}")
        print(f"Age : {self.age}")
        print(f"Course : {self.course}")

    def intro(self):                         # another method
        print(f"Hello, I am {self.name}.")
        print(f"I am {self.age} years old and I am studying {self.course}.")

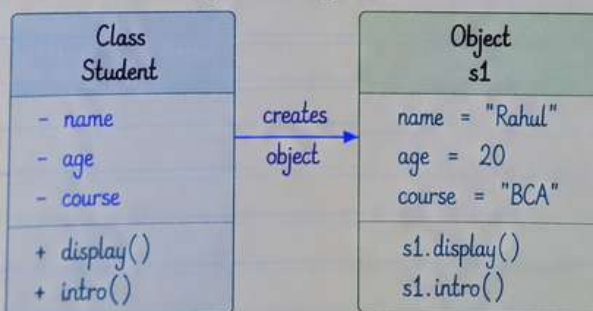
# Creating object of class
s1 = Student("Rahul", 20, "BCA")            # object creation

# Calling methods using object
s1.display()                                # method call
print("-----")
s1.intro()
```

3. Real World Analogy

- जैसे किसी College में Student के पास name, age, course जैसी information होती है ।
- उसी तरह हम Student class बनाकर इन properties को model कर सकते हैं ।
- हर student (object) की अपनी अलग values हो सकती हैं लेकिन सभी एक ही class के होते हैं ।

4. Class and Object Diagram



5. Output

Output :

Name : Rahul
Age : 20
Course : BCA

Hello, I am Rahul.

I am 20 years old and I am studying BCA.

6. Explanation

1. class Student : यहाँ Student नाम की class बनाई गई है ।
2. __init__() : यह constructor होता है जो object बनाने समय automatically call होता है और initial values set करता है ।
3. self : यह current object को refer करता है ।
4. self.name, self.age, self.course : ये instance variables हैं जो object के data को store करते हैं ।
5. display() and intro() : ये instance methods हैं जो object के behavior को define करते हैं ।
6. s1 = Student(...) : यहाँ Student class का object बनाया गया है ।
7. s1.display() और s1.intro() : object के methods को call करके output प्राप्त किया जाता है ।

Key Point :

- Class एक blueprint (स्काफ़) की तरह होती है ।
- Object class का instance होता है ।
- __init__() constructor object initialize करने के लिए उपयोग होता है ।
- Instance variables object के data को store करते हैं ।
- Instance methods object के behavior को define करते हैं ।

7. Advantages

- Code reusability (एक class से कई objects बनाए जा सकते हैं) ।
- Real world modeling में मदद करता है ।
- Data और functions को एक साथ organize करता है ।
- Program को modular और easy to maintain बनाता है ।

8. Conclusion

इस example से हमने देखा कि class in Python का उपयोग करके हम real world entities (जैसे Student) को आसानी से model कर सकते हैं । Class, object, constructor, instance variables और methods मिलकर Python को एक powerful और flexible programming language बनाते हैं ।

Topic : Program 1 of Class in Python

(Question and Solve)

1. Question :

Create a class named Student with the following attributes and methods :

- Attributes : name, age, course
- Method 1 : `display()` : to show the student details
- Method 2 : `change_course()` : to update the course of the student

Create an object of the class, call the methods and show the output.

2. Approach / Logic :

- सबसे पहले Student नाम की class बनाएँगे।
- `__init__()` constructor में name, age और course की initial values लेंगे और instance variables में store करेंगे।
- `display()` method द्वारा student का detail show करेंगे।
- `change_course()` method द्वारा course को update करेंगे।
- Class का object बनाएँगे और methods को call करेंगे।

3. Program (Solution) :

```
class Student:
    def __init__(self, name, age, course):
        self.name = name
        self.age = age
        self.course = course
```

```
    def display(self):
        print("Name :", self.name)
        print("Age :", self.age)
        print("Course :", self.course)
```

```
    def change_course(self, new_course):
        self.course = new_course
        print("Course updated successfully!")
```

Creating object

```
s1 = Student("Rahul", 20, "BCA")
```

Displaying initial details

```
print("---- Student Details (Before) ----")
s1.display()
```

Changing course

```
s1.change_course("MCA")
```

Displaying updated details

```
print("---- Student Details (After) ----")
s1.display()
```

4. Output :

--- Student Details (Before) ---

Name : Rahul

Age : 20

Course : BCA

Course updated successfully!

--- Student Details (After) ---

Name : Rahul

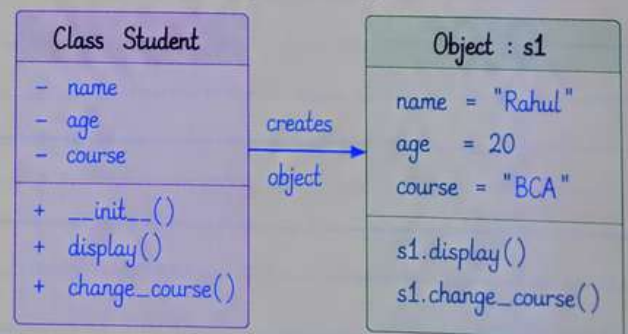
Age : 20

Course : MCA

5. Explanation :

1. `class Student` : यह एक class है जो student की information को represent करती है।
2. `__init__()` : यह constructor होता है जो object बनते समय automatically call होता है और name, age, course को initialize करता है।
3. `self` : यह current object को refer करता है।
4. `display()` : यह method student के details को print करता है।
5. `change_course()` : यह method course को update करता है।
6. `s1 = Student(...)` : इससे class का object बनता है।
7. `s1.display()` और `s1.change_course()` : object के methods को call करके output प्राप्त किया जाता है।

6. Class and Object Diagram :



7. Key Points :

- Class एक blueprint (खाका) की तरह होता है।
- Object class का instance होता है।
- `__init__()` constructor object initialize करने के लिए उपयोग होता है।
- Instance variables object के data को store करते हैं।
- Methods object के behavior को define करते हैं।
- Program को और useful बनाने के लिए इसमें और भी methods जोड़े जा सकते हैं।